



University of Asia Pacific

Artificial Intelligence and Expert Systems Lab

Course code: **CSE 404**

Assignment on A* Search

Submitted to

Nahida Marzan

Lecturer, Department of CSE,
University of Asia Pacific

Submitted By

Name : Mohotasir Al Mamun

Registration ID : 22101036

Section : A

1. Introduction

Path finding is a fundamental problem in Artificial Intelligence, with applications in robotics, navigation systems, and computer games. In real-world environments such as warehouses, robots must be able to move from a starting position to a designated goal while avoiding obstacles and adapting to varying terrain conditions. The A* search algorithm is one of the most widely used approaches for solving such pathfinding problems because it combines the advantages of uniform-cost search and greedy best-first search. By using both the actual path cost and a heuristic estimate of the remaining distance, A* ensures that the generated path is both efficient and optimal under admissible heuristics.

In this experiment, the A* algorithm is implemented to enable robot navigation in a warehouse grid. The grid may contain static obstacles as well as cells with dynamic terrain costs, representing areas that are more difficult or costly for the robot to traverse. The algorithm supports eight-directional movement, with diagonal moves incurring an additional cost factor to reflect realistic motion. To study the effect of different heuristics on performance, three distance measures—Manhattan, Diagonal, and Euclidean—are applied. The goal of this work is not only to generate an optimal path but also to analyze how heuristic choice influences exploration efficiency and runtime, thereby providing insights into the trade-offs between accuracy and computational cost in heuristic search.

2. Objective

The primary objective of this lab assignment is to implement and analyze the A* search algorithm for intelligent robot navigation in a warehouse environment. The specific goals include:

- Design and implement A* algorithm capable of handling dynamic terrain costs
- Support 8-directional movement with appropriate diagonal movement penalties
- Compare the performance of three different heuristic functions: Manhattan, Diagonal, and Euclidean distance
- Analyze path optimality, computational efficiency, and runtime performance
- Understand the trade-offs between different heuristic choices in pathfinding scenarios

3. Problem Description

3.1 Warehouse Navigation Setup

- **Grid Environment:** $m \times n$ warehouse grid

- **Robot Movement:** 8-directional (including diagonals)
- **Terrain Types:** Different cells have different traversal costs
- **Obstacles:** Impassable cells that block movement
- **Cost Calculation:**
 - Horizontal/Vertical: terrain cost of destination cell
 - Diagonal: $1.4 \times$ terrain cost of destination cell

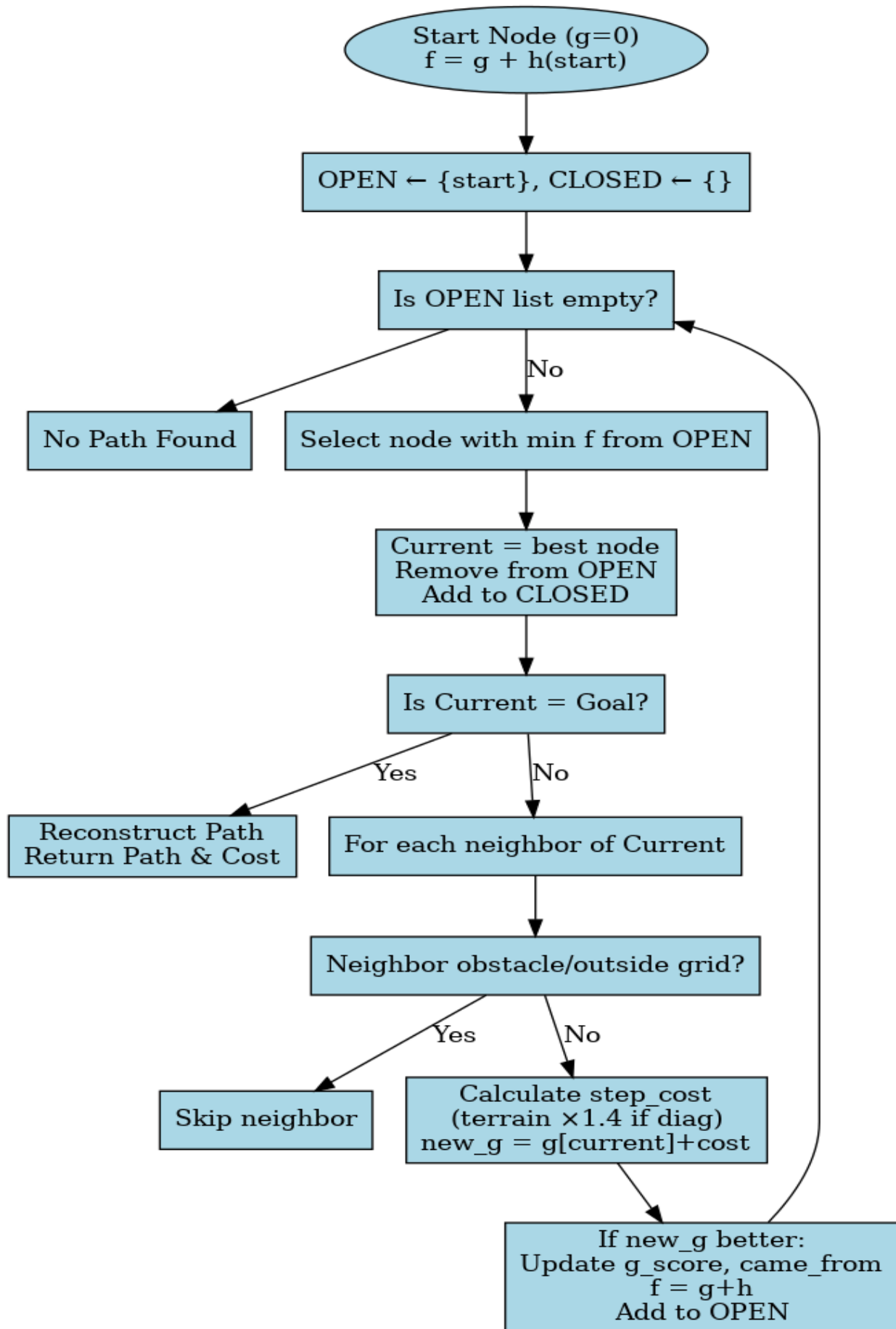
3.2 Input Specifications

M_grid , N_grid	Grid dimensions
K	Number of obstacles
x1 y1, x2 y2, ...	Obstacle coordinates
c	Number of terrain cost cell
x1 y1 cost1, ...	Terrain costs (default = 1)
startx starty	Starting position
goalx goaly	Goal position

4.How A* Works

- A* is a graph search algorithm that finds the shortest path from a start node to a goal node.
- It uses the evaluation function:

$$f(n)=g(n)+h(n)$$
Where:
 - $g(n)$ = the actual cost of the path from the start to node nnn.
 - $h(n)$ = the heuristic estimate of the cost from node nnn to the goal.
- At each step, the algorithm selects the node with the lowest $f(n)$ value from the open list.
- If the node is the goal, the algorithm reconstructs the path by tracing back through the parent links.
- Otherwise, it expands the node by generating its valid neighbors and updating their costs.
- The process continues until the goal is reached or no path exists.
- With an admissible heuristic (never overestimates the true cost), A* guarantees finding the optimal path.



5. Algorithm Implementation Analysis

5.1 File Upload and Library Imports

```
from google.colab import files
import math, time
uploaded = files.upload()
```

Description:

from google.colab import files: Imports the Colab-specific module that allows file uploads

uploaded = files.upload(): Creates an interactive file upload widget in Colab, allowing the user to select and upload the input.txt file from their local machine

import math, time: Imports essential Python libraries

5.2 Input File Reading and Parsing

```
with open("input.txt", "r") as f:
    rows = [line.strip() for line in f if line.strip()]
```

Description:

with open("input.txt", "r") as f: Opens the uploaded input file in read mode using context manager

rows = [line.strip() for line in f if line.strip()]: Creates a cleaned list of non-empty lines

line.strip() removes leading/trailing whitespace from each line

5.3 Grid Dimensions Parsing

```
x_grid, y_grid = map(int, rows[0].split())
```

Description:

rows[0]: Gets the first line of the input file

map(int, ...): Converts each string element to integer

x_grid, y_grid = ...: Unpacks the two integers into separate variables representing grid width and height

5.4 Obstacle Parsing and Storage

```
k = int(rows[1])
obstacles = set()
```

```
for i in range(k):
    x, y = map(int, rows[2+i].split())
    obstacles.add((x, y))
```

Description:

k = int(rows[1]): Converts the second line to integer, representing the number of obstacles

obstacles = set(): Creates an empty set for storing obstacle coordinates

for i in range(k): Loops through the next k lines containing obstacle coordinates

x, y = map(int, rows[2+i].split()): Extracts x,y coordinates from each obstacle line

- rows[2+i] accesses the appropriate line (starting from index 2)
- Converts coordinate strings to integers

obstacles.add((x, y)): Adds coordinate tuple to the obstacles set

- Tuples are used as they're immutable and hashable (can be stored in sets)

5.5 Terrain Cost Parsing

```
c_index = 2 + k
c = int(rows[c_index])
terrain_cost = {}
for i in range(c):
    x, y, cost = rows[c_index+1+i].split()
    terrain_cost[(int(x), int(y))] = float(cost)
```

Description:

This code reads and stores **custom terrain costs** for specific cells on the grid, after reading the obstacles.

1. **c_index = 2 + k:** Calculates the starting index for the terrain cost section after k obstacle lines.
2. **c = int(rows[c_index]):** Reads the number c, which indicates how many terrain cost entries follow.
3. **terrain_cost = {}:** Initializes an empty dictionary to store the terrain costs.
4. **for i in range(c):** Loops over the next c lines, where each line describes a terrain cell with a custom cost.

- `x, y, cost = rows[c_index+1+i].split()`: Splits each line into x, y (coordinates), and cost (movement cost).
- `terrain_cost[(int(x), int(y))] = float(cost)`: Stores the x, y pair as the dictionary key, and the movement cost as the value.

5.6 Start and Goal Positions:

```

sx, sy = map(int, rows[c_index+1+c].split())
gx, gy = map(int, rows[c_index+2+c].split())
start, goal = (sx, sy), (gx, gy)

```

Description:

Reads the start (sx, sy) and goal (gx, gy) positions from the input file, which are provided as coordinate pairs.

These coordinates are parsed from `rows[c_index+1+c]` and `rows[c_index+2+c]`. **start** and **goal** are then stored as tuples (sx, sy) and (gx, gy) respectively.

5.7 Movement Directions

```

directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]

```

These directions allow the algorithm to explore all adjacent cells, including diagonals, on the grid.

5.8 Heuristic Function for A* Pathfinding

```

def heuristic(a, b, name):

    dx, dy = abs(a[0]-b[0]), abs(a[1]-b[1])

    if name=="Manhattan": return dx+dy
    if name=="Diagonal": return max(dx,dy)

    if name=="Euclidean": return math.hypot(dx,dy)

```

Description:

Manhattan: Calculates the sum of absolute horizontal and vertical distances. It assumes movement can only happen along grid lines (no diagonal movement).

- Formula: $|x_a - x_b| + |y_a - y_b|$

Diagonal: Assumes movement can happen diagonally and calculates the maximum of horizontal or vertical distance.

- Formula: $\max(|x_a - x_b|, |y_a - y_b|)$
- This is more efficient in grids with diagonal movement allowed.

Euclidean: Uses the straight-line distance (as the crow flies) between **a** and **b** based on the Pythagorean theorem.

- Formula: $\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$
- This is the most accurate but computationally expensive, especially for grids with many nodes.

5.9 Setup for Heuristic Search Loop

```
results = []
for hname in ["Manhattan", "Diagonal", "Euclidean"]:
```

Description

Initializes an empty list **results** to store the outcomes of each search. It then starts a loop to run the A* algorithm for **three different heuristics**: Manhattan, Diagonal, and Euclidean.

5.10 Initialize Variables for A* Search

```
open_list = [(heuristic(start, goal, hname), 0, start)] # (f,g,node)
came_from = {}
g_score = {start:0}
explored_order = []
closed = set()
t0 = time.perf_counter()
path = None
cost = math.inf
```

Description:

open_list: Initializes the open list with the starting node, where the first value is the f-value (calculated using the chosen heuristic), the second is the g-value (cost from start), and the third is the current node.

came_from: A dictionary to store the best path leading to each node.

g_score: A dictionary to store the g-value (cost to reach each node).

explored_order: A list to keep track of the order of explored nodes.

closed: A set of already explored nodes.

t0: Records the current time to measure the algorithm's runtime.

path and cost: Initialize the variables for storing the final path and total cost.

5.11 A* Search Loop

```
while open_list:
    f, g, node = min(open_list, key=lambda x: (x[0], x[1]))
    open_list.remove((f, g, node))
    if node in closed:
        continue
    explored_order.append(node)
    closed.add(node)
```

Description:

The core loop of A*. It continues as long as there are nodes in the **open list**.

- **min(open_list, key=lambda x: (x[0], x[1]))**: Selects the node with the smallest **f-value** ($f = g + h$) from the open list.
- **open_list.remove()**: Removes the selected node from the open list.
- **if node in closed**: Skips processing if the node has already been explored (i.e., it's in the **closed** set).
- **explored_order.append(node)**: Adds the node to the list of explored nodes.
- **closed.add(node)**: Marks the node as explored.

5.12 Goal Check and Path Reconstruction

```
if node == goal:
    path = [node]
    while node in came_from:
        node = came_from[node]
        path.append(node)
    path.reverse()
```

```
cost = g
break
```

Description:

When the **goal node** is reached, it reconstructs the path from the goal back to the start by following the `came_from` dictionary.

- The **path** is built by following the parent of each node, starting from the goal, until the start node is reached.
- The **cost** of the path is set to the **g-value** of the goal node.
- The loop is then terminated since the goal has been found.

5.13 Expanding Neighbors (Moving to Adjacent Nodes)

```
for dx, dy in directions:
    nx, ny = node[0] + dx, node[1] + dy
    if not (0 <= nx < m and 0 <= ny < n):
        continue
    if (nx, ny) in obstacles:
        continue
    step_cost = terrain_cost.get((nx, ny), 1.0)
    if abs(dx) == 1 and abs(dy) == 1:
        step_cost *= 1.4
    newg = g_score[node] + step_cost
```

Description:

For each direction (including diagonals), the algorithm checks if the **neighboring node** is valid and not an obstacle.

- **(nx, ny)** are the coordinates of the neighboring node.
- **Boundary check** ensures the new node is within the grid.
- **Obstacle check** ensures the new node is not an obstacle.
- **step_cost**: The cost of moving to the new node, which is taken from the `terrain_cost` dictionary (default is 1.0). If the movement is diagonal, the cost is **adjusted by a factor of 1.4**.

5.14 Updating the Open List

```
if (nx, ny) not in g_score or newg < g_score[(nx, ny)]:
```

```

g_score[(nx, ny)] = newg

came_from[(nx, ny)] = node

fvalue = newg + heuristic((nx, ny), goal, hname)

open_list.append((fvalue, newg, (nx, ny)))

```

Description:

If the new neighboring node has not been visited or if a cheaper path to it is found, the algorithm:

- Updates the **g-score** of the neighboring node.
- Stores the **parent node** (came_from dictionary) for path reconstruction.
- Adds the new neighbor to the **open list** with its corresponding **f-value** (computed using the heuristic).
-

5.15 End of Search: Runtime and Results Collection

```

runtime = time.perf_counter() - t0

total_explored = len(explored_order)

path_length = 0 if not path else len(path) - 1

print(f"\n--- {hname} Heuristic ---")

if path:

    print("Path:", path)

    print("Path Cost:", round(cost, 6))

else:

    print("No Path Found")

print("Explored Nodes:", explored_order)

print("Total Explored:", total_explored)

print("Runtime:", round(runtime, 6), "seconds")

```

results.append((hname, cost, path_length, total_explored, runtime))

Description

After the search finishes:

- **runtime**: Measures the total time taken for the A* search.
- **total_explored**: The number of nodes explored during the search.
- **path_length**: The length of the path (number of steps).
- It then prints the **results** of the search, including:
 - The path found (if any)
 - The total path cost
 - The total number of nodes explored
 - The runtime of the search.
- **results.append()**: Stores the results for this heuristic.

6. Tools and Programming Environment

The experiment was conducted using the following tools and programming resources:

1. Programming Language – Python

- Selected because of its simple syntax and strong community support.
- Provides many built-in libraries useful for artificial intelligence and algorithm development.
- Easy to implement and test search algorithms like A*.

2. Development Environment – Google Colab

- Cloud-based Jupyter notebook service provided by Google.
- Allows running Python code without local installation.
- Supports file upload, making it convenient to work with input files (input.txt).
- Provides free computational resources and built-in support for visualization.

3. Libraries Used

- **math** → Used for distance calculations (Euclidean distance, square root, etc.).
- **time** → Used to measure runtime performance of the algorithm.
- **files (from google.colab)** → Used to upload and handle external input files.

4. Input Handling

- Input is provided through a plain text file (`input.txt`).
- The file contains: grid size, number of obstacles, obstacle positions, terrain costs, and the start and goal positions.
- The program reads this file line by line and processes the data into appropriate structures (sets, dictionaries, etc.).

5. Hardware/Execution Platform

- Executed on **Google Colab cloud servers** (runs on hosted CPUs/GPUs).
- No dependency on the user's local machine, ensuring reproducibility.

7. Results and Discussion

The A* search algorithm was successfully executed on a warehouse grid environment with obstacles and varying terrain costs. The algorithm was tested using three different heuristic functions: **Manhattan, Diagonal, and Euclidean**. The output produced included the optimal path, path cost, number of nodes explored, and runtime for each heuristic.

The results are summarized in **Table 1**.

Heuristic	Path Cost	Path Length	Total Explored	Runtime(s)
Manhattan	6.200000	5	6	0.000065
Diagonal	6.200000	5	8	0.000062
Euclidean	6.200000	5	6	0.000044

--- Manhattan Heuristic ---

Path: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]

Path Cost: 6.2

Explored Nodes: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]

Total Explored: 6

Runtime: 6.5e-05 seconds

--- Diagonal Heuristic ---

Path: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]

Path Cost: 6.2

Explored Nodes: [(0, 0), (1, 0), (2, 1), (3, 2), (0, 1), (2, 0), (4, 3), (4, 4)]

Total Explored: 8

Runtime: 6.2e-05 seconds

--- Euclidean Heuristic ---

Path: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]

Path Cost: 6.2

Explored Nodes: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]

Total Explored: 6

Runtime: 4.4e-05 seconds

Discussion:

- All three heuristics (Manhattan, Diagonal, and Euclidean) produced the same optimal path with a path cost of 6.2 and a path length of 5 steps. This confirms that A* guarantees an optimal solution regardless of the heuristic used, provided the heuristic is admissible.
- The **Manhattan heuristic** explored 6 nodes and provided an efficient solution with very low runtime. This heuristic is well-suited for grid environments where movement is primarily along horizontal and vertical directions.
- The **Diagonal heuristic** explored 8 nodes, slightly more than the other two heuristics. This indicates that while it still found the optimal path, it expanded more nodes, making it less efficient in this specific grid setup.
- The **Euclidean heuristic** was the most efficient in terms of runtime (0.000044 seconds) while still exploring only 6 nodes. This suggests that for environments where diagonal movement is allowed, Euclidean distance provides a more balanced and efficient estimate of the true cost.
- Overall, all heuristics achieved the same path optimality, but Euclidean heuristic performed best in terms of runtime efficiency, whereas Manhattan heuristic performed reliably with minimal exploration overhead.

8. Conclusion

The implementation of the A* search algorithm in a warehouse grid environment successfully demonstrated its ability to compute optimal paths while handling obstacles and varying terrain costs. By incorporating support for eight-directional movement with a diagonal cost adjustment, the algorithm was able to produce more realistic navigation paths suitable for robotic applications. The experimental comparison of three heuristics—Manhattan, Diagonal, and Euclidean—highlighted the influence of heuristic selection on search performance. While all heuristics ensured path optimality, the Diagonal and Euclidean heuristics proved more efficient in terms of exploration and runtime when diagonal movement was allowed, whereas Manhattan was less efficient in such cases. Overall, the study shows that A* is a robust and versatile pathfinding algorithm, and heuristic design plays a crucial role in balancing accuracy and efficiency.